

# Sıfırdan Next.js.

*Boş bir klasörden canlı yayına, yirmi bir adımda.*

Bu kitap bir eğitim serisinden değil, **gerçek bir projeden** çıktı. Buradaki her komut yazıldı, her hata gerçekten alındı, her tuzağa gerçekten düşüldü. Düşülen yerler **kırmızı kutularda** duruyor — çünkü en çok orada öğrendik.

21 ADIM

8 BÖLÜM

15 ŞEKİL

12 TUZAK

21 KONTROL SORUSU · CEVAPLI

HTML SÜRÜMÜNDE ŞEKİLLER OYNUYOR

GİT · NODE · TYPESCRIPT · NEXT.JS · POSTGRES · VERCEL

# İçindekiler

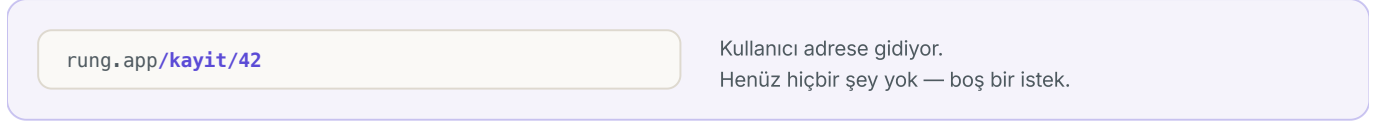
|    |                             |                                       |
|----|-----------------------------|---------------------------------------|
| 01 | Nasıl okunur                | iki dakika                            |
| 02 | Terminal ve Git             | klasörden depoya                      |
| 03 | Node, npm ve paketler       | JavaScript tarayıcının dışında        |
| 04 | TypeScript                  | yazarken yakalayan katman             |
| 05 | Next.js'in iskeleti         | klasör, adres, JSX                    |
| 06 | Next.js — tam harita        | yönlendirme · sunucu/istemci · render |
| 07 | Sunucu tarafı ve veritabanı | Server Components                     |
| 08 | Yayın                       | git push = canlı                      |

Ekler: tek sayfa harita · komut sözlüğü · terim sözlüğü · on iki tuzak · cevaplar

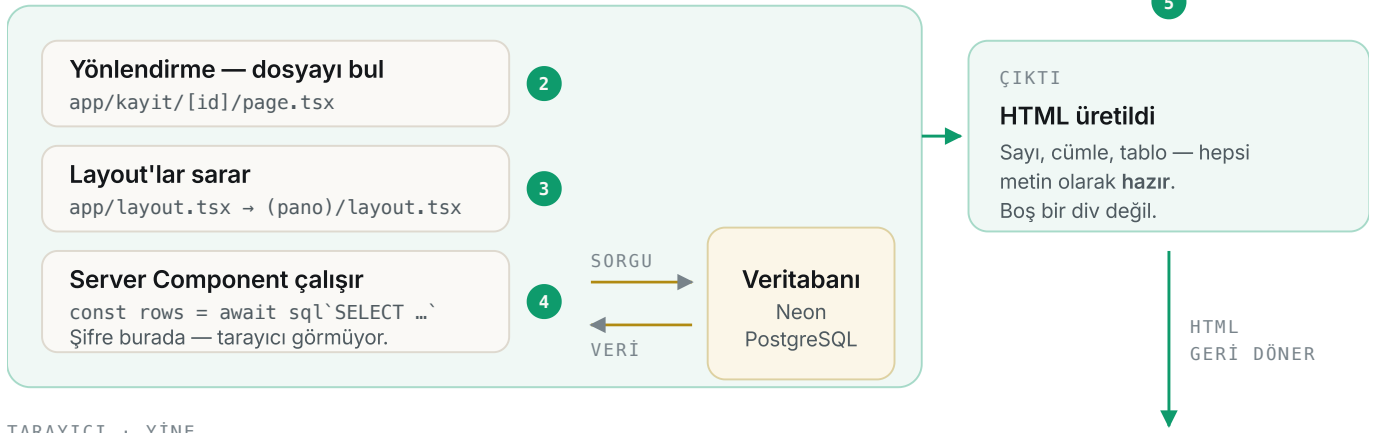
# Tek sayfa: bir isteğin yolculuğu

Kitabın tamamı bu resmin ayrıntısı. Şimdi anlamasan da olur — **her bölümden sonra buraya dön**, her seferinde bir kutu daha yerine oturacak.

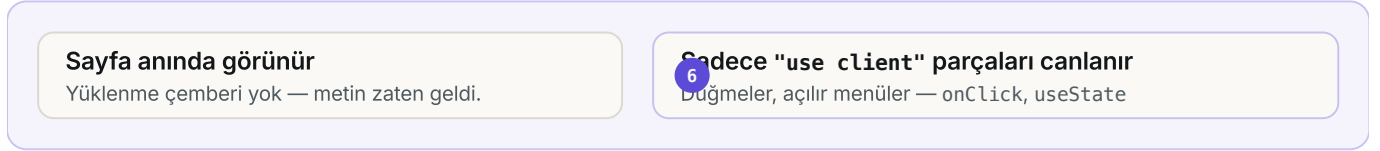
TARAYICI



SUNUCU • VERCEL • FRANKFURT



TARAYICI • YİNE



**Altı adım, tek cümle:** tarayıcı adresi ister • Next.js klasör ağacından dosyayı bulur • layout'lar sarar • sunucuda veri çekilir • **HTML hazır gider** • tarayıcıda sadece işaretli parçalar canlanır. Klasik React'te 2–5 arası *tarayıcıda* olurdu; fark tam olarak bu.

□ tarayıcı — herkes görebilir   □ sunucu — kimse göremez   □ veritabanı

# Nasıl okunur

Bu kitabı okuyarak öğrenemezsin. Yazarak öğrenirsin.

Kitabın çıktığı projede tek bir kural vardı: **kavramı anlatan başkası, kodu yazan sen**. Sebebi basit — izleyerek öğrenilmiyor, çünkü takılan sen olmuyorsun.

O yüzden her komutu kendin yaz. Kopyala-yapıştır bile yapma, harfleri tuşla. Yavaş, sinir bozucu, ve öğrenmenin gerçekten olduğu tek yer.

## — Kutuların anlamı

### • İPUCU

İşe yarayan bir detay, bir alışkanlık, ya da bilinen bir gelenek.

### • TUZAK

Buraya düşülüyor. Bu kitaptaki tuzakların hepsine **gerçekten düşüldü**.

### • TANIM

Bir terim ilk geçtiğinde burada tanımlanıyor. "Bunu zaten biliyorsundur" cümlesi yok.

### • KURAL

Ezberlenecek değil, anlaşılacak ilke. Genelde bir cümle.

## — Varsayımlar

HTML, CSS ve JavaScript biliyorsun. **Başka hiçbir şey varsayılmıyor** — terminal, git, npm, TypeScript, React, veritabanı, dağıtım; hepsi sıfırdan.

Bilgisayarında Node.js kurulu olmalı. Bir de GitHub hesabı. Gerisi yol boyunca.

## — Bir bölümü nasıl çalışırsın

Kitap baştan sona okunmak için değil, **bölüm bölüm çalışılmak** için yazıldı. Her bölümde aynı dört hareket:

- 01 Şekle bak, metni sonra oku.** Şekiller özet değil, **anlatının kendisi**. Çoğu bölümde resim tek başına yarısını söylüyor.
- 02 Komutu kendi elinle yaz.** Kopyalama. Yazarken yaptığın yazım hatası, hata mesajı okumayı öğrendiğin yerdir.
- 03 Bir şey patlayınca dört soruyu sor:** hangi satır · hangi tür hata · ne bekliyordu · ne buldu. Düzeltmeye ancak dördü cevaplanınca geç.
- 04 Bölüm sonundaki üç soruyu cevapla.** Kafanda değil, **sesli cümleyle**. Kuramıyorsan bölüm oturmamış demektir — geri dön.

### • KURAL

Tekrar okumak başarısızlık değil, **yöntem**. Bu kitabın çıktığı projede `layout.tsx` üç ayrı anlatımda oturdu — üçüncüsünde. Normal olan bu.

Sessizce yanlış olan kod, gürültülü şekilde çöken koddan çok daha tehlikelidir.

# Terminal ve Git

Klasörden depoya, depodan GitHub'a. Kod yazmadan önceki her şey.

## — Neden terminal

Kullanacağın araçların çoğunun düğmesi yok — git'in yok, npm'in yok, veritabanı bağlantısının yok. Bu bir eksiklik değil, tercih.

Sebebi şu: **tıklamayı kimseye gönderemezsin**. "Şuraya tıkla, sonra oraya" diye anlatmak zorunda kalırsın. Ama `git init -b main` satırını kopyalayıp yapıştırabilirsin — ve altı ay sonra kendine de yapıştırabilirsin.

### • TERMİNAL

Bilgisayara yazıyla komut verdiğin pencere. *Komut satırı* veya *konsol* da deniyor.

Tarayıcıda F12 → Console'a `document.title` yazarsın, cevap verir. Terminal aynı şey — ama tek bir sayfayla değil, **bütün bilgisayarla** konuşuyorsun.

## — Çalışma klasörü

Terminal her an **bir klasörün içinde duruyor**. Yazdığın komutlar oraya uygulanıyor. Bir odada durmak gibi: "ışığı aç" dediğinde bulunduğun odanın ışığı açılır.

### DOĞRU

Desktop/

```
└─ rung/
   └─ app/
   └─ docs/
```

terminal burada

```
$ git init -b main
```

Sadece proje takibe girer.  
İstedğimiz bu.

### YANLIŞ

Desktop/

```
└─ rung/
   └─ ekran-goruntusu.png
   └─ vergi-belgesi.pdf
```

terminal burada

```
$ git init -b main
```

Masaüstündeki **her şey** takibe girer.  
Geri almak için `.git`'i elle sil.

**Şekil 1.** Aynı komut, iki farklı klasör, iki farklı sonuç. Git komutu yazmadan önce `pwd` yaz.

```
$ pwd
/Users/adin/Desktop/rung
```

`pwd` = **p**rint **w**orking **d**irectory

## — Sürüm kontrolü

Herkesin ilk çözümü aynıdır:

```
style.css
style-yedek.css
style-son.css
style-son-GERCEK.css
```

Bu çalışmıyor. Üç şeyi bilmiyorsun: hangisi neydi, **neyi neden** değiştirdin, ve iki dosyada birden çalıştıysan hangi ikisinin uyumlu olduğunu.

**Sürüm kontrolü**, projenin zaman içindeki her hâlini notuyla birlikte saklayan sistem. En yaygını **Git**. Oyunlardaki kayıt noktası gibi — ama kayıtların hepsi duruyor, her birinin notu var, ve aralarındaki farkı satır satır görebiliyorsun.

```
$ git init -b main
Initialized empty Git repository in .../rung/.git/
```

**init** *initialize* — bu klasörü depoya çevir

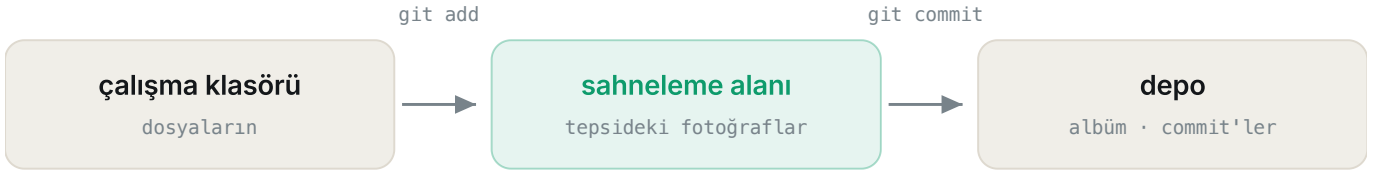
**-b main** *branch* — ana dalın adı `main` olsun

### • NEDEN -B MAIN

Yazmazsan Git dalı `master` diye adlandırır ve bir uyarı basar. `main` bugün standart; sonradan değiştirmek fazladan iş. Baştan doğrusunu yaz.

## — Git'in üç bölgesi

Git'te "kaydet" tek komut değil, iki. Sebebi şu: bir öğleden sonra iki iş yapmış olabilirsin — bir hata düzelttin **ve** renkleri değiştirdin. Tek komut olsaydı ikisi aynı kayda girerdi, birini geri alırken diğeri de giderdi.



commit = fotoğraf · add = kareye kimin gireceğine karar vermek

**Şekil 2.** Sahneleme alanı bir liste değil, **bir yer** — içinde dosya adları değil, dosya *içerikleri* duruyor.

• TUZAK · BURADA TAKILDIK

**git add** bir işaret koymuyor. Dosyanın o andaki **içeriğinin kopyasını** alıyor.

Sonuç: **git add** yaptıktan sonra dosyayı tekrar düzenlersen, o düzeltme commit'e **girmez**.

Tepsideki fotoğrafta hâlâ eski hâli var. Tekrar **git add** gerekiyor.

Böyle bir durumda **git status** aynı dosyayı **iki bölümde birden** gösterir.

## — Ne kaydedilmez

**.DS\_Store** dosyasını sen oluşturmadın — macOS Finder her klasörü açtığında bırakıyor. Depoya koyarsan iki sorun: **her açışta değiştiği için** gerçek değişikliklerin arasında gürültü yapar, ve ikinci bir kişi katılırsa kimsenin umursamadığı bir dosya için çakışma çıkar.

• KURAL

Depo, projenin **ne olduğunu** tutar — senin bilgisayarının arkasında bıraktığı izleri değil.

Genellemesi: **üretilen şey depoya girmez, üreten şey girer.**



## .GITIGNORE

```
# macOS'un bıraktığı dosyalar
.DS_Store

# Gizli anahtarlar – asla depoya girmez
.env
.env.local

# Bağımlılıklar – package.json'dan geri gelir
node_modules/

# Next.js'in derleme çıktısı
.next/
out/
```

### • EN PAHALI TUZAK

`.gitignore` **sadece takip edilmeyen** dosyalara uygulanır. Bir dosya bir kez takibe girdiyse, sonradan eklemek onu **durdurmaz** — `git rm -r --cached` gerekir.

Ve geçmiş yine dolu kalır: her `git clone` o veriyi indirmeye devam eder. Gerçekten silmek geçmişini yeniden yazmayı gerektirir, bütün commit kimlikleri değişir.

`.gitignore` **koruma, tedavi değil**. Bu yüzden `.env` satırı, o dosya daha *doğmadan önce* yazılır. Bir anahtar bir kez commit edilirse yanmıştır.

## — Commit mesajı

Commit atmak kolay, zor olan mesajı yazmak. Altı ay sonra `update`, `fix`, `son hali`, `asdasd` gibi bir liste sana hiçbir şey söylemez. Ve bir işverenin repona bakınca gördüğü ilk şey bu liste.

**Conventional Commits** bir yazım standardı. Kural tek satır: `tip: ne yaptığın`

|          |                                       |
|----------|---------------------------------------|
| feat     | Yeni bir özellik eklendi              |
| fix      | Bir hata düzeltildi                   |
| docs     | Sadece belge değişti                  |
| refactor | Davranış aynı, kod yeniden düzenlendi |
| test     | Test eklendi veya değişti             |
| chore    | Yapılandırma, araç, kurulum           |

#### • KURAL

Mesaj İngilizce, küçük harfle başlar, **emir kipiyle** yazılır. "added" değil `add`. Sebebi: commit, "*bu commit uygulandığında ne olur*" sorusunun cevabıdır.

**Ve bir commit bir iş yapar.** İki iş varsa hedefli `git add dosya-adı` ile ikiye böl — sonra birini geri almak isteyince diğeri yerinde kalır.

#### • TUZAK

`-m` yazmazsan git bir metin editörü açar — muhtemelen **vim**, ve çıkmayı bilmezsen orada sıkışırsın. Kurtuluş: `:q!` + Enter.

## — Uzak depo

Bütün geçmişin tek bir diskte duruyor. İki sıkıntı: **disk ölürse aylar uçar**, ve **kimse göremez** — kimsenin göremediği portfolyo, portfolyo değil.

#### • GİT ≠ GİTHUB

**Git** bilgisayarında çalışan program, internet gerektirmez. **GitHub** git depolarını barındıran bir şirket — git'in parçası değil, sadece en popüler ev sahibi.



Biri "asıl", diğeri "kopya" değil — **ikisi de tam**. Git dağıtıktır.

**Şekil 3.** GitHub hesabın silinse projen kaybolmaz. Bütün geçmiş `.git` klasöründe de duruyor.

#### • -U BAYRAĞI

`git push -u origin main` — `-u = --set-upstream` : "yereldeki `main` , `origin` 'deki `main` 'i takip etsin."

**Dal başına bir kez.** Sonra `git push` yeter. Ama yeni bir dal açarsan onun için tekrar gerekir.

#### GERÇEKTEN BAŞIMIZA GELDİ

"Deploy ettim ama site değişmedi"

Kod yazıldı, commit atıldı, Vercel'de deploy tetiklendi — ama canlıda eski sayfa duruyordu. Yarım saat kaybettik.

Cevap `git status` 'un **ilk satırındaydı**:

```
## main...origin/main [ahead 2]
```

**ahead 2** = "yerelde iki commit var, GitHub'da yok." Vercel GitHub'dan yayınlıyor, senin diskinden değil.

**Ders:** `git commit` ≠ **yayın**. `git push` = **yayın**. Ve `git status` 'un ilk satırı bunu her zaman söylüyor.

## Bu bölüm oturdu mu?

Cevabı kafanda **cümleyle** kur, sonra kitabın sonundaki cevaplara bak. Üçünü de bilemiyorsan bölümü tekrar oku — tekrar başarısızlık değil, yöntem.

- 1 `git add index.html` yazdın, sonra aynı dosyada bir satır daha değiştirdin ve `git commit` attın. Son değişiklik `commit`'e girdi mi?
- 2 GitHub yarın kapansa projen gider mi?
- 3 `.env.local` 'i yanlışlıkla `commit`'ledin, fark edip `.gitignore` 'a ekledin. Şifren güvende mi?

# Node, npm ve paketler

JavaScript'i tarayıcının dışına çıkarmak — ve neden mecburuz.

## — *Neden sunucu gerekiyor*

Bir web uygulamasının üç işi tarayıcıda yapılamaz:

- Veritabanına bağlanmak — şifre gerekiyor
- Bir API'ye anahtarla istek atmak — anahtar gerekiyor
- Başka kullanıcıların verisini gizlemek — "gizle" diyen kodun kullanıcıda olmaması gerekiyor

Ortak nokta: hepsinde **kullanıcının görmemesi gereken bir şey** var.

### • TUZAK

Tarayıcıya gönderdiğin her satır kodu kullanıcı okuyabilir — F12 yeter. Bir API anahtarını oraya koyarsan biri kopyalar, senin hesabından binlerce istek atar, fatura sana gelir.

**Kodu karıştırıp gizlemek işe yaramaz:** tarayıcı o anahtarı kullanabiliyorsa, bir insan da onu çıkarabilir. Tek çözüm anahtarın tarayıcıya hiç gitmemesi.

## — *Node.js*

JavaScript'i tarayıcının dışına çıkarıp doğrudan bilgisayarda çalıştıran program.

### TARAYICIDA

`document` , `window` `var`

Dosya okuyamazsın

Veritabanına bağlanamazsın

Kodun herkese açık

### NODE'DA

**yok** — sayfa diye bir şey yok

**okursun**

**bağlanırsın**

**kodun kimseye görünmez**

**Yeni bir dil değil** — bildiğin JavaScript, farklı bir yerde.

## — npm ve paketler

**Paket** = başkasının yazıp paylaştığı, kurup kullanabileceğin kod parçası. `npm install <paket>` yazınca npm onu internetten indirip `node_modules/` klasörüne koyuyor ve `package.json` 'a bir satır yazıyor.

**Kurmak** = başkasının kodunu projenin içine indirmek. Sihir yok.



Listeyi ve fişi saklarsın. Poşetleri değil — `npm install` onları her zaman yeniden doldurur.

**Şekil 4.** Üç paket istedik, **19 klasör** geldi — bağımlılıkların bağımlılıkları. Kilit dosyası hangi sürümün kurulduğunu tam olarak yazıyor; "bende çalışıyordu" problemi böyle çözülüyor.

### • DEPENDENCIES VS DEVDEPENDENCIES

Ölçüt tek soru: **canlıdaki sunucu buna ihtiyaç duyuyor mu?**

`react` **çalışıyor** → `dependencies`. `typescript` sadece **kontrol ediyor**, derleme bitince yerde kalıyor → `devDependencies` (`npm install -D`).

## — Neden hazır kurulum aracı kullanmadık

Next.js'in `create-next-app` aracı tek komutla otuz dosyalık çalışan bir uygulama üretiyor. Bu kitapta **kullanmadık**.

Sebebi: o otuz dosyayı anlamazsın. Çalışan bir şeyin olur ama içinde ne olduğunu bilmezsin, bir şey bozulduğunda nereye bakacağını da bilmezsin. Beş adım daha uzun sürüyor; karşılığında her satırın neden orada olduğunu biliyorsun.

## GERÇEKTEN BAŞIMIZA GELDİ

```
"type": "commonjs"
```

`npm init` bir soru soruyor: `type (commonjs)` . Enter'a bastık, kaldı.

İlk `.tsx` dosyasını yazdığımızda:

```
Error: Specified module format (CommonJs) is not matching  
the module format of the source code (EcmaScript Modules)
```

`package.json` "bu proje CommonJS" diye beyan etmişti; dosyada ise `import` vardı. Paketleyici itiraz etti.

**Çözüm:** `type` satırını **tamamen sil**. Next.js'in kendi kurulum aracı da yazmıyor — alan yoksa paketleyici dosya bazında karar veriyor, çakışacak bir beyan olmuyor.

## • KENDİNİ SINA

### Bu bölüm oturdu mu?

Cevabı kafanda **cümleyle** kur, sonra kitabın sonundaki cevaplara bak. Üçünü de bilemiyorsan bölümü tekrar oku — tekrar başarısızlık değil, yöntem.

- 1 `node_modules/` klasörünü sildin, proje çalışmaz oldu. Bu klasör depoya girmiyorsa nasıl geri geliyor?
- 2 `typescript` neden `devDependencies` 'te ama `@neondatabase/serverless` `dependencies` 'te?
- 3 Tarayıcıda `document.title` yazınca cevap alıyorsun, Node'da yazınca hata. Dil mi farklı?

# TypeScript

Çalışmayan, sadece bakan bir katman. Ve tam da bu yüzden değerli.

## — Sessiz yanlış

```
const density = entry.errors / entry.wordCount * 100;
```

`entry.wordCount` bir sayı mı? Veritabanından string olarak gelmiş olabilir. Öyleyse `"74" / 4` → **18.5**. Çalışır, hata fırlatmaz, ve yanlışdır.

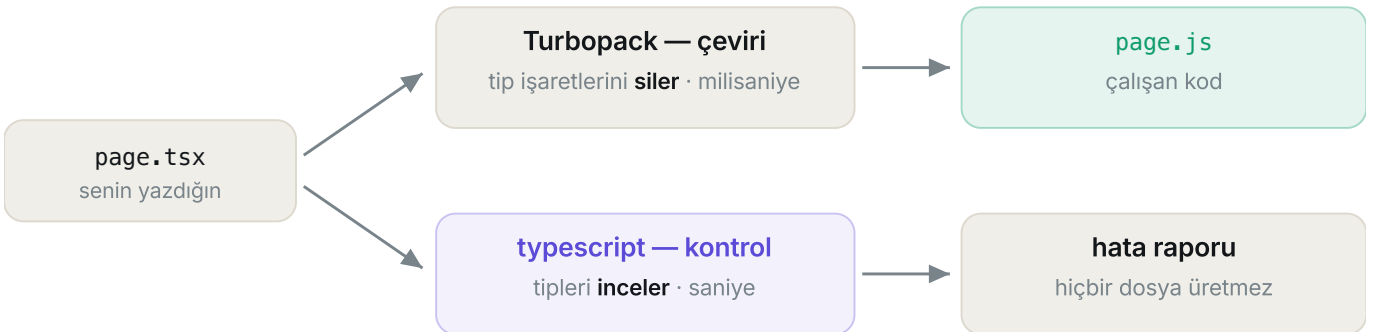
Çöken kodu düzeltirsin. **Sessizce yanlış olan sayıyı aylarca fark etmezsin.**

## — Ne olduğu, ne olmadığı

TypeScript, JavaScript'in üzerine **tip** ekleyen katman. Yeni bir dil değil — her geçerli JavaScript, geçerli TypeScript.

### • EN ÖNEMLİ CÜMLE

**TypeScript çalışmaz.** Sunucuya ve tarayıcıya giden şey derlenmiş JavaScript; tip işaretleri derlemede siliniyor. Tek satırı bile canlıya gitmiyor.



**Şekil 5.** İki iş, iki araç. Tip **silmek** aptal ve hızlı bir iş; tip **kontrol etmek** zeki ve yavaş. Her tuşa bastığında ikincisini beklemek istemezsin — o yüzden ayrılmışlar.



Ne zaman hangisi devreye giriyor: editörde sürekli kontrol · `npm run dev` tip hatasında **durmaz**, sadece uyarır · `npm run build` **durur**. Yani bozuk tiple canlıya çıkamıyorsun.

- @TYPES/... PAKETLERİ

React düz JavaScript ile yazılmış, içinde tip bilgisi yok. `@types/react` onun **kullanma kılavuzu** — içinde çalışan tek satır kod yok, sadece tanım.

Bu yüzden `react` `dependencies` 'te ama `@types/react` `devDependencies` 'te: biri çalışıyor, diğeri sadece anlatıyor.

## — Katı mod

`tsconfig.json` 'da `"strict": true` . Next.js varsayılan olarak `false` koyuyor — mevcut bir projeye eklendiğinde yüzlerce hata fırlatmasın diye. **Yeni bir proje için doğru varsayılan değil.**

İki şeyi yakalıyor:

- **Tipsiz parametre.** `function f(x)` → *"x implicitly has an 'any' type"*
- **null kontrolü.** `el.textContent` → *"el is possibly null"*. JavaScript'te en sık çökme sebebi buydu; katı mod onu **sen yazarken** yakalıyor.

- DARALTMA – NARROWING

`process.env.X` 'in tipi **her zaman** `string | undefined` , çünkü TypeScript `.env` dosyasının içeriğini bilmiyor.

```
const url = process.env.DATABASE_URL; // string | undefined
if (!url) throw new Error("...");      // boşsa burada duruyor
const sql = neon(url);                  // string
```

TypeScript kodun **akışını** takip ediyor: "boş olsaydı üstte durmuştu, demek ki dolu."

! koyup susturmak da mümkündü — **yapma**. Katı modu açıp susturmak anlamsız. Yan fayda: değişken eksikse anlamsız çökme yerine " `.env.local` 'i kontrol et" yazan bir hata alıyorsun.

## Bu bölüm oturdu mu?

Cevabı kafanda **cümleyle** kur, sonra kitabın sonundaki cevaplara bak. Üçünü de bilemiyorsan bölümü tekrar oku — tekrar başarısızlık değil, yöntem.

- 1 `npm run dev` tip hatasında durmuyor, `npm run build` duruyor. Bu neden kasıtlı?
- 2 `process.env.DATABASE_URL` 'in tipini TypeScript neden `string | undefined` sayıyor?
- 3 Tip bilgisi canlıdaki koda hiç girmiyorsa TypeScript sana ne kazandırıyor?

# Next.js'in iskeleti

Klasör yapısının adres olduğu yer, ve neden dosya adları sabit.

## — Kütüphane ve çerçeve

HTTP sunucusu, yönlendirme, HTML üretme, derleme — hepsi **çözülmüş** problemler. **Çerçeve** bunların hazır geldiği yapı.

### • TEK CÜMLELİK FARK

**Kütüphaneyi sen çağırırsın, çerçeve seni çağırır.** Sen kuralına göre dosya koyarsın, o ne zaman çalıştıracağını bilir.

Klasik React'ta `index.js` 'i sen yazar, `ReactDOM.render()` 'ı sen çağırırdın. Next.js'te o kodu Next.js üretiyor ve `.next/` klasörüne yazıyor.

`.next/` = **senin elle yazacağın bağlantı kodunun durduğu yer.** Zaten bu yüzden `.gitignore` 'da: üretilen şey depoya girmez.

## — App Router — klasör adresin kendisi

Ayrı bir rota listesi yok. Klasör yapısı zaten liste.

KLASÖR

ADRES

app/

├ layout.tsx

HER SAYFADA

├ page.tsx

→ /

├ gecmis/page.tsx

→ /gecmis

└ kayit/[id]/page.tsx

→ /kayit/42

Rota listesi yok — **liste klasör yapısının içinde.**

**Şekil 6.** `page.tsx` ve `layout.tsx` özel isimler. Bu altı isim dışındaki her dosya Next.js için görünmez — onlar senin, kendin import edersin.

- NEDEN İSİMLER SABİT

Next.js klasörü tarıyor ve "hangisi sayfa, hangisi kabuk" diye soruyor. İsimler serbest olsa ya dosyaları açıp tahmin etmesi ya da sana bir liste yazdırması gerekirdi.

**Dosyanın adı, o listenin kendisidir.**

Özel isimler: `page` · `layout` · `loading` · `error` · `not-found` · `route` .

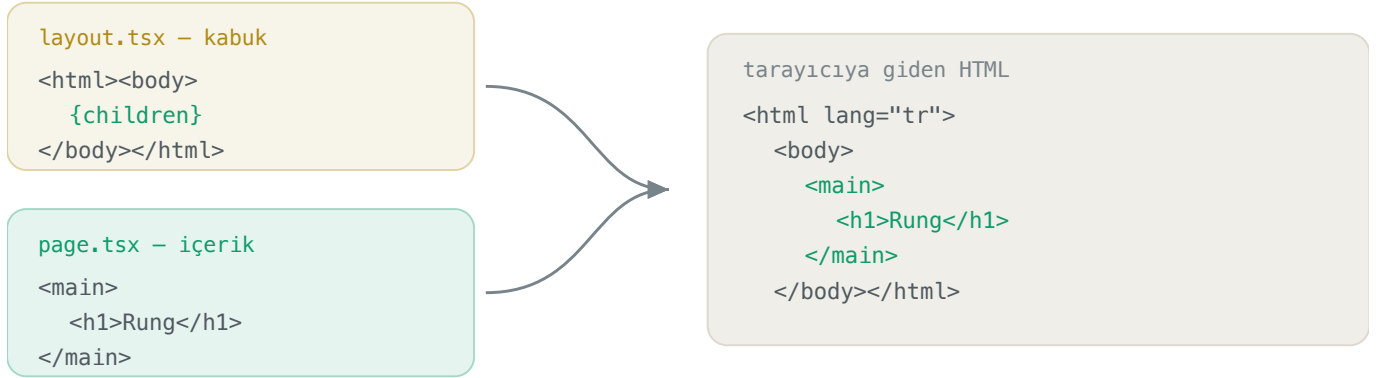
- HANGİ İSİM ZORUNLU, HANGİSİ SERBEST

`export default` → **isim serbest**. Next.js "varsayılan"ı alıyor, adına bakmıyor.

`export const metadata` → **isim zorunlu**. Adıyla arıyor. `sayfaBilgisi` deseydin sessizce görmezden gelirdi.

## — Kabuk ve içerik

Her sitede iki tür parça var: her sayfada **tekrar eden** (üst bar, logo) ve o sayfaya **özel** olan. Tekrar edeni her dosyaya kopyalarsan, logoyu değiştirmek beş dosya düzenlemek olur.



Layout hangi sayfada olduğunu bilmiyor — sadece **boş yer** bırakıyor.

**Şekil 7.** Next.js önce `Page()` 'i çalıştırıyor, çıkan sonucu `children` olarak layout'a veriyor. Resim çerçevesi gibi: çerçeve içine hangi fotoğrafın geleceğini bilmiyor.

## — JSX

JavaScript'in içinde HTML gibi görünen sözdizimi. Üç farkı var:

`className`

`class` JavaScript'te ayrılmış kelime

Tek kök element

İki kardeş etiketi yan yana döndüremezsin

`{ }`

"Buraya JavaScript değeri koy"

#### • JSX ASLINDA YOK

Paketleyici `<h1>Rung</h1>` yazdığın şeyi `jsx("h1", { children: "Rung" })` hâline getiriyor. **JSX, fonksiyon çağırısının güzel görünen hâli.**

#### GERÇEKTEN BAŞIMIZA GELDİ

### Yorumun sayfada görünmesi

Canlıdaki sayfada şu yazı duruyordu:

```
Veritabanı saati: Fri Aug 14 2026 ... // ← 3. ekrana bas
```

Açıklama yorumu JSX'in içine yazılmıştı. **JSX'in içinde** `//` **yorum değildir** — etiketlerin arasındaki her şey metin sayılır ve ekranda görünür.

**Doğrusu:** `{/* böyle */}` . JavaScript kısmındaki `//` normal çalışır — fark, JSX'in içinde mi dışında mı olduğu.

#### • KENDİNİ SINA

### Bu bölüm oturdu mu?

Cevabı kafanda **cümleyle** kur, sonra kitabın sonundaki cevaplara bak. Üçünü de bilemiyorsan bölümü tekrar oku — tekrar başarısızlık değil, yöntem.

- 1 `page.tsx` 'i hiçbir yerden import etmiyorsun, yine de sayfa ekrana geliyor. Onu kim çağırıyor?
- 2 `export default function Sayfa()` — adını `Sayfa` koydun, çalıştı. Peki `export const metadata` yerine `export const sayfaBilgisi` yazsan?
- 3 Aynı üst bar üç sayfada duruyor ve sekme değiştirtince yanıp sönmüyor. Bunu hangi dosya sağlıyor?

# Next.js — tam harita

Bu bölüm anlatı değil, **harita**. Kitabın geri kalanı yaşananları anlatıyor; burası Next.js'in bütün temel kavramlarını sıfırdan, tek yerde topluyor.

## • ÖNCE ŞUNU AYIRALIM

Karşına çıkan her terim Next.js terimi değil. Üç ayrı sözlük var:

**Next.js:** `page` · `layout` · Server/Client Component · `"use client"` · server action · statik/dinamik render

**Veritabanı:** şema · **göç (migration)** · sorgu · indeks · yabancı anahtar

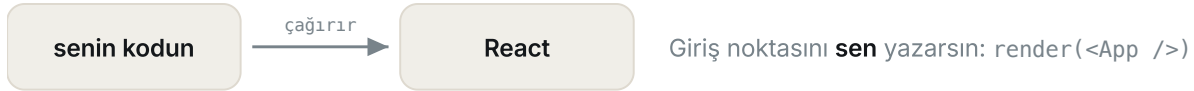
**Web:** çerez · oturum · hash · HTTP · port

`migration` bir Next.js terimi **değil**. Doğru sözlükte aramak, aramanın yarısı.

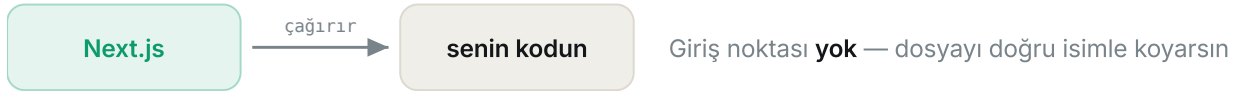
## — 1· Kontrol kimde

Her şeyin dayandığı tek fark bu. **Kütüphane** senin çağırdığın koddur; **çerçeve** seni çağıran koddur.

### KLASİK REACT — KÜTÜPHANE



### NEXT.JS — ÇERÇEVE



**Şekil A.** Ok yön değiştirdi. Next.js'te sabit dosya adlarının sebebi bu — seni **bulması** gerekiyor.

## — 2· Yönlendirme — klasör adresin kendisi

Rota listesi diye bir dosya yok. Klasör ağacının kendisi liste.

| DOSYA                                     | ADRES                  | NE                                            |
|-------------------------------------------|------------------------|-----------------------------------------------|
| <code>app/page.tsx</code>                 | <code>/</code>         | Anasayfa                                      |
| <code>app/gecmis/page.tsx</code>          | <code>/gecmis</code>   | Klasör adı = yol parçası                      |
| <code>app/kayit/[id]/page.tsx</code>      | <code>/kayit/42</code> | <b>Dinamik parça</b> — köşeli parantez        |
| <code>app/(pano)/ilerleme/page.tsx</code> | <code>/ilerleme</code> | <b>Grup</b> — parantezli klasör adrese girmez |

- KÖŞELİ PARANTEZ NE İŞE YARIYOR

`[id]` yazan klasör, **o konumdaki her değeri kabul ediyor**. `/kayit/42` ve `/kayit/117` aynı dosyaya düşüyor; hangi sayı geldiği sayfaya parametre olarak veriliyor. Rung'da 142 kayıt için 142 dosya yazmayacağız — **tek dosya**, değişen kısım köşeli parantez.

### — 3·Layout — iç içe kabuklar

`layout.tsx` sadece kökte olmak zorunda değil. **Her klasöre bir tane koyabilirsin**, ve iç içe geçerler.

`app/layout.tsx` — `<html>` `<body>` · her sayfada

`app/(pano)/layout.tsx` — üst bar, sekmeler · sadece pano sayfalarında

`app/(pano)/ilerleme/page.tsx`

#### Sayfanın kendi içeriği

Grafikler, sayılar — sadece burada

**Şekil B.** Sayfa değişince **sadece en içteki kutu** yeniden çiziliyor. Üst bar yerinde kalıyor — bu yüzden sekme değiştirmek anında hissediliyor.

### — 4·Server ve Client Components

Bir sayfada iki zıt ihtiyaç var: **veritabanına erişmek** (şifre gerekiyor, kullanıcıya gitmemeli) ve **düğmeye tepki vermek** (kullanıcının parmağı orada, kod onda olmalı).

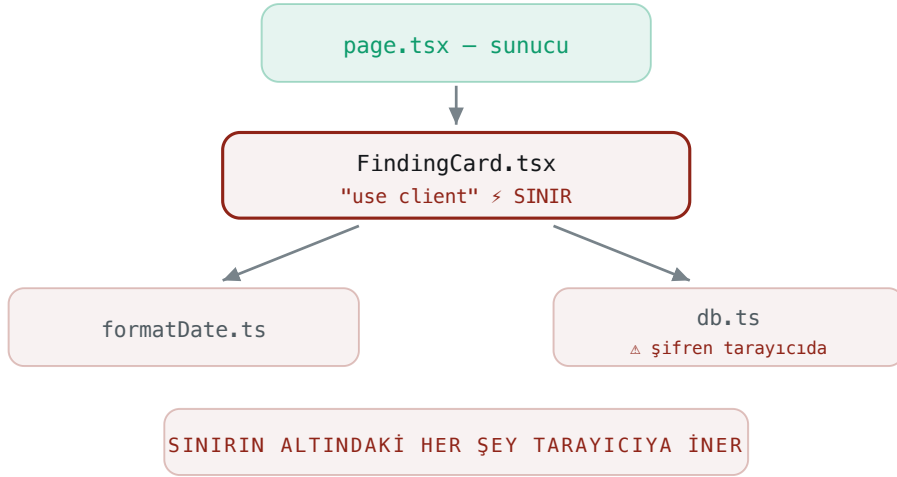
Next.js varsayılanı **sunucu** yaptı. Tarayıcıda çalışması gerekeni sen işaretliyorsun:

```
// dosyanın EN ÜSTÜ, her şeyden önce
"use client";
```

#### • EN ÇOK YANLIŞ ANLAŞILAN YER

**"use client" bir düğme değil, bir SINIR.**

İşaretlediğin dosya tek başına gitmiyor — **o dosyanın import ettiği her şey de** tarayıcıya iniyor. Ve onların import ettikleri de.



**Şekil C.** Bu yüzden kural şu: **sınırı olabildiğince aşağıya it.** Yaprğa yakın tut, köke değil — sayfanın tamamını değil, sadece düğmeyi işaretle.

#### "USE CLIENT" İLE KAZANILAN

onClick , onChange — etkileşim

useState — hafıza tutmak

useEffect — tarayıcı olayları

#### KAYBEDİLEN

async bileşen olamaz

Veritabanına doğrudan erişemez

Gizli anahtar kullanamaz

#### • RUNG'DA NASIL OLACAK

Analiz ekranı: solda metin, sağda bulgu kartları, her kartta "Katılmıyorum" düğmesi.

app/analiz/page.tsx → **sunucu**, bulguları veritabanından çeker

FindingCard.tsx → **"use client"**, sadece düğme mantığı

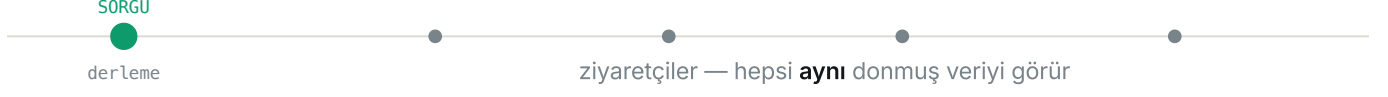
Sayfanın tamamını istemciye taşısaydık iki şey bozulurdu: veritabanı sorgusu orada çalışamazdı, ve bağlantı dizesi tarayıcıya inerdi.



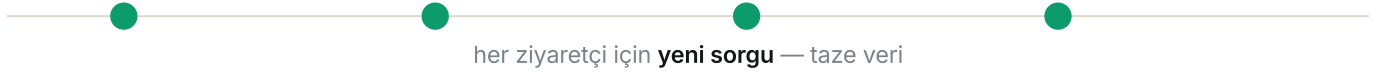
## — 5·Render biçimi — ne zaman çalışıyor

Aynı kod iki farklı anda çalışabiliyor, ve bu fark her şeyi değiştiriyor.

STATİK — derlemede bir kez



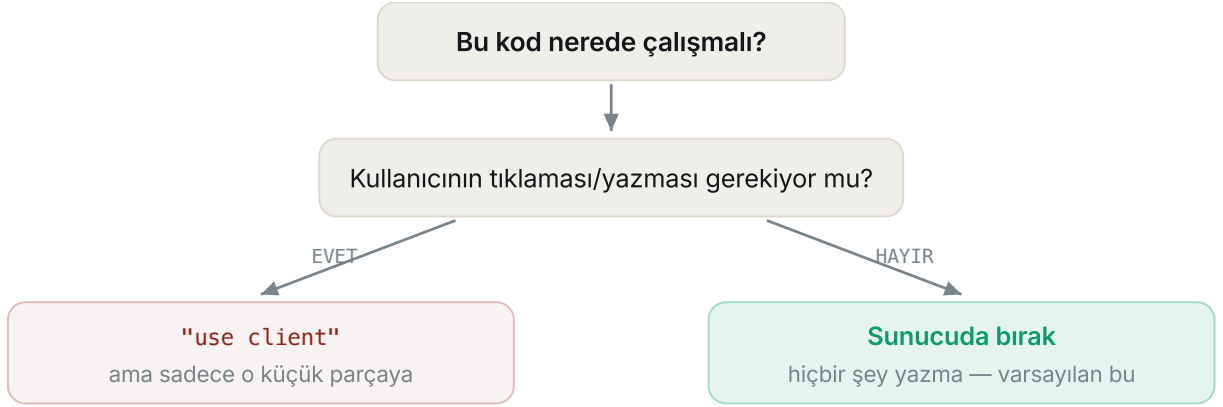
DİNAMİK — her istekte



**Şekil D.** Statik sayfa hızlıdır ama **donar**. Kullanıcı verisi gösteren her sayfa dinamik olmalı: `export const dynamic = "force-dynamic"`.

## — 6·Karar ağacı

Yeni bir dosya yazarken sorman gereken tek soru dizisi:



Şüphedeysen **sunucuda bırak**. Etkileşim lazım olunca alta iner, işaretlersin.

**Şekil E.** Aşama 02'den itibaren her yeni dosyada bu soruyu soracağız.

• BÖLÜMÜN ÖZETİ – BEŞ CÜMLE

1. Çerçeve seni çağırır, o yüzden dosya adları sabit.
2. Klasör ağacı adres listesidir; `[köşeli]` değişken parça, `(parantez)` adrese girmez.
3. `layout` iç içe geçer; sayfa değişince sadece en içteki kutu yenilenir.
4. Varsayılan sunucu. `"use client"` bir sınırdır — altındaki her şey tarayıcıya iner.
5. Kullanıcı verisi gösteren sayfa **dinamik** olmalı, yoksa veri donar.

• KENDİNİ SINA

## Bu bölüm oturdu mu?

Cevabı kafanda **cümleyle** kur, sonra kitabın sonundaki cevaplara bak. Üçünü de bilemiyorsan bölümü tekrar oku — tekrar başarısızlık değil, yöntem.

- 1 `app/kayıt/[id]/page.tsx` — 142 kayıt için kaç dosya yazacaksın?
- 2 Bir dosyanın en üstüne `"use client"` yazdın. O dosya `db.ts` 'i import ediyor. Ne oldu?
- 3 Sayfa veritabanından saat gösteriyor ama saat hep aynı kalıyor. Teşhis?

# Sunucu tarafı ve veritabanı

Backend ve frontend nasıl tek dosyada birleşti — ve neyi kaybettik.

## — Server Components

Klasik React'ta **her şey tarayıcıda** çalışır. Veritabanına bağlanmak isteseydin ayrı bir arka uç yazman, ondan API açman, tarayıcıdan ona istek atman gerekirdi. Üç parça.

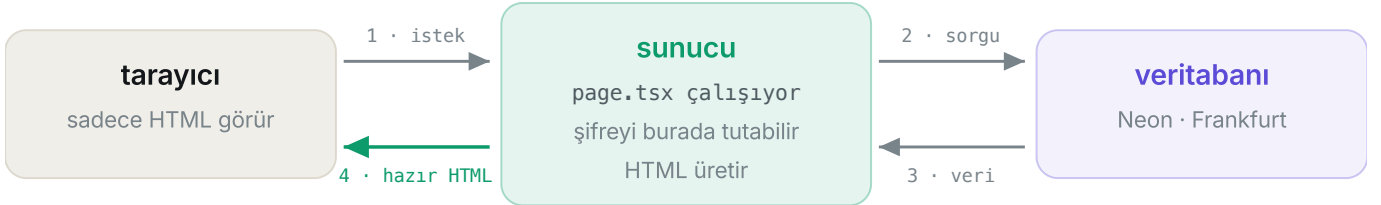
Next.js App Router'da bileşenler **varsayılan olarak sunucuda** çalışıyor.

### • NASIL KANITLANIR — OTUZ SANİYE

Bir sayfa bileşenine `console.log("merhaba")` koy ve sayfayı aç.

Mesaj **tarayıcı konsolunda değil, terminalde** çıkıyor. Çünkü fonksiyon sunucuda çalıştı, HTML üretti, ve tarayıcıya **sadece HTML** gitti.

İkinci kanıt: sayfa kaynağına bak — içerik **hazır** duruyor. Klasik React'ta boş bir `<div id="root">` görürdün.



Tarayıcı veritabanıyla **hiç konuşmadı**.  
Bileşenin kodu tarayıcıya **hiç ulaşmadı**.

**Şekil 8.** Ara katman kaybolmadı — **onu yazma işi kayboldu**. Eskiden `/api/entries` rotasını, JSON'u, `fetch`'i ve yükleniyor durumunu sen yazardın.

- SUNUCU MU, TARAYICI MI

Next.js "her şey sunucuda" demiyor — **varsayılanı** sunucu yapıyor. Tarayıcıda çalışması gereken dosyanın başına `"use client"` yazıyorsun.

**Ölçüt:** kullanıcının etkileşimi gerekiyor mu? Tıklama, yazma, sekme değiştirme → tarayıcı. Veri çekme, gizli anahtar, HTML üretme → sunucu.

## — *Ne kazandık, ne kaybettik*

### KAZANÇ

Sözleşme senkronu bitti — eskiden bir alan eklemek dört yeri güncellemekti

İlk yüklemde boş ekran ve spinner yok

Tipler uçtan uca akıyor

### KAYIP

Ayrı ölçekleme yok — ikisi birlikte büyüyor

Mobil uygulama gelirse gerçek bir API yine lazım

**Sınır görünmezleşti** — eskiden bir URL'di, şimdi bir satır

Sonuncusu en önemlisi: sınır ucuzladı ama görünmezleşti. Gizli bir anahtarı istemci tarafına sürüklemek eskisinden kolay. Çerçeve seni koruyor, ama disiplin sende.

## — *Gizli bilgi*

Bağlantı dizesi tek satırda **adres + kullanıcı adı + şifre** taşıyor. `.env.local` dosyasında duruyor ve `.gitignore` 'da.

- KURAL

Gizli bilgi kodla değil, **her ortamla ayrı ayrı** taşınır. Senin makinende `.env.local`, sunucuda panel. Kod ikisinde de aynı: `process.env.DATABASE_URL`.

- NEXT\_PUBLIC\_ ÖN EKİ

Next.js ortam değişkenlerini **varsayılan olarak sadece sunucuda** gösteriyor. Adı `NEXT_PUBLIC_` ile başlıyorsa derleme sırasında **kodun içine metin olarak yapııştırıp** tarayıcıya yolluyor.

Yani ön ek, bilinçli bir **"yayınla" onayı**. Kazara sızdıramazsın, ancak isteyerek sızdırırsın.

## Bu bölüm oturdu mu?

Cevabı kafanda **cümleyle** kur, sonra kitabın sonundaki cevaplara bak. Üçünü de bilemiyorsan bölümü tekrar oku — tekrar başarısızlık değil, yöntem.

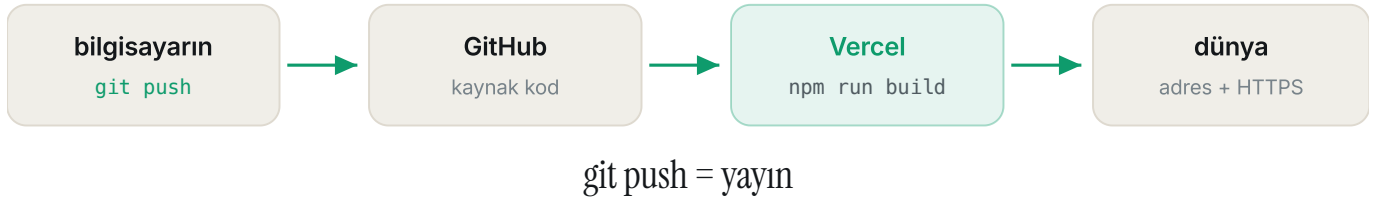
- 1 Server Component'e koyduğun `console.log` nerede çıkar — ve bu neden bütün mimariyi özetler?
- 2 Klasik React'te sayfanın kaynağında `<div id="root">` boş görünürdü, şimdi içerik dolu. Kullanıcı için pratik farkı ne?
- 3 `NEXT_PUBLIC_` ön ekini koymadıkça ortam değişkeni tarayıcıya gitmiyor. Bu ön ek neyi zorunlu kılıyor?

# Yayın

localhost'tan internete. Ve sondaki iki tuzak.

`localhost` = "bu bilgisayar". Laptop kapanınca site yok. Başkasının görmesi için 7/24 açık, adresi ve HTTPS sertifikası olan, Node çalıştırabilen bir makine gerekiyor.

Eskiden: sunucu kiral, Linux kur, nginx ayarla, sertifika al, kod değişince elle güncelle. Günler süren, ayrı bir uzmanlık isteyen iş.



**Şekil 9.** Vercel depoyu izliyor. Push'u görünce kodu çekiyor, derliyor, dağıtıyor. Başka hiçbir şey yapmıyorsun.

## GERÇEKTEN BAŞIMIZA GELDİ

**password authentication failed**

Ortam değişkeni panele girilmişti, ama derleme çöküyordu:

```
Error [NeonDbError]: password authentication failed
for user 'neondb_owner'
```

Hata çok şey söylüyordu: değişken **vardı** (yoksa bizim `throw` 'umuz çalışırdı), adres **doğruydu**, kullanıcı adı **tanınmıştı**. Sadece şifre yanlıştı.

Sebebi: `.env.local` 'de değer tırnak içinde yazılıyor — `DATABASE_URL="postgres://..."`. O tırnaklar **dosya biçiminin parçası**, Next.js onları soyuyor. Panelin form kutusu ise dosya değil: ne yapıştırdıysan harfi harfine alıyor.

**Aynı metin, iki farklı kural.** Dosyada tırnak gerekir, panelde gerekmez.

## — Statik mi, dinamik mi

Next.js varsayılan olarak sayfayı **derlemede bir kez** üretiyor. Çoğu sayfa için doğru: bir kez üret, dağıt, veritabanına hiç dokunma. Çok hızlı.

Ama kullanıcı verisi gösteren sayfada bu yanlış — veri donuyor.

|                  | STATİK                 | DİNAMİK          |
|------------------|------------------------|------------------|
| Sorgu ne zaman   | Derlemede, bir kez     | Her istekte      |
| Doğru olduğu yer | Tanıtım sayfası, belge | Kullanıcı verisi |

### • SİNSİ TUZAK

Statik sayfa **çalışıyor gibi görünür** — veri var, sayfa açılıyor. Ama gösterdiği şey derleme anından kalma.

**Teşhis:** sayfayı iki kez yenile. Değişmiyorsa statik. Ya da `npm run build` çıktısında `o` (Static) ara. Ya da yanıt başlığına bak: `x-vercel-cache: HIT` → statik, `MISS` → dinamik.

**Çözüm:** `export const dynamic = "force-dynamic";`

Bir yan etkisi daha var: statikken sorgu derlemede çalıştığı için ortam değişkeni **build anında** gerekiyor. Yoksa derleme çöküyor ve eski sürüm servis edilmeye devam ediyor.

Boş bir klasörden başladın. Şimdi internette bir adres var, açınca sayfa geliyor, ve sayfadaki sayı her istekte başka bir kıtadaki veritabanından geliyor.

### • KENDİNİ SINA

#### Bu bölüm oturdu mu?

Cevabı kafanda **cümleyle** kur, sonra kitabın sonundaki cevaplara bak. Üçünü de bilemiyorsan bölümü tekrar oku — tekrar başarısızlık değil, yöntem.

- 1 Commit attın, Vercel hâlâ eski sayfayı gösteriyor. İlk nereye bakarsın?
- 2 `.env.local` 'de `DATABASE_URL="postgres://..."` yazıyor. Aynısını tırnaklarıyla Vercel paneline yapıştırırsan ne olur?
- 3 `localhost:3000` sende açılıyor, arkadaşında açılmıyor. Neden?

# Sözlükler ve tuzaklar

Kavramı unutursan bölüme dön; komutu unutursan buraya.



## — Komutlar

|                                                |                                                           |
|------------------------------------------------|-----------------------------------------------------------|
| <code>pwd</code>                               | Hangi klasördeyim                                         |
| <code>cd ~/Desktop/rung</code>                 | Klasör değiştir · <code>~</code> = kullanıcı klasörün     |
| <code>git init -b main</code>                  | Klasörü depoya çevir, ana dal <code>main</code>           |
| <code>git status</code>                        | Git ne görüyor. Zararsız, istediğin kadar yaz             |
| <code>git status -sb</code>                    | İlk satır: <code>ahead N</code> = push edilmemiş N commit |
| <code>git add . / git add a b</code>           | Hepsini / sadece belirtilenleri sahnele                   |
| <code>git commit -m "tip: mesaj"</code>        | Kayıt noktası oluştur                                     |
| <code>git remote add origin &lt;url&gt;</code> | Uzak depoyu tanı                                          |
| <code>git push -u origin main</code>           | İlk gönderim — dal başına bir kez                         |
| <code>git check-ignore -v &lt;dosya&gt;</code> | Bu dosyayı hangi kural yok sayıyor                        |
| <code>git revert &lt;kimlik&gt;</code>         | O commit'in tersini yapan yeni commit                     |
| <code>node --version</code>                    | Node kurulu mu, hangi sürüm                               |
| <code>npm init</code>                          | <code>package.json</code> oluştur                         |
| <code>npm install &lt;paket&gt;</code>         | Paket kur → <code>dependencies</code>                     |
| <code>npm install -D &lt;paket&gt;</code>      | Geliştirme paketi → <code>devDependencies</code>          |
| <code>npm run dev</code>                       | Geliştirme sunucusu · durdur: <b>Ctrl+C</b>               |
| <code>npm run build</code>                     | Canlıya hazır derleme · <b>tip hatasında durur</b>        |
| <code>npx tsc --noEmit</code>                  | Sadece tip kontrolü, dosya üretme                         |

## — Terimler

|                  |                                                                     |
|------------------|---------------------------------------------------------------------|
| repository       | Git'in takip ettiği klasör. Kısaca <b>repo</b>                      |
| commit           | Kayıt noktası — fotoğraf + neden çekildiğini anlatan not            |
| branch           | Paralel çalışma hattı                                               |
| staging area     | Commit'e girecek içeriklerin toplandığı ara bölge                   |
| untracked        | Git'in gördüğü ama takip etmediği dosya                             |
| remote           | Deponun başka makinedeki kopyası. Genelde <code>origin</code>       |
| upstream         | Yerel dalın takip ettiği uzak dal. <code>-u</code> ile kurulur      |
| package          | Başkasının yazıp paylaştığı kod parçası                             |
| dependency       | Projenin <b>çalışmak için</b> ihtiyaç duyduğu paket                 |
| framework        | Çözülmüş problemlerin hazır geldiği yapı                            |
| App Router       | Klasör yapısının adres yapısı olduğu model                          |
| JSX              | JavaScript içinde HTML gibi yazılan sözdizimi                       |
| Server Component | Sunucuda çalışıp HTML üreten bileşen. <b>Varsayılan</b>             |
| Client Component | Tarayıcıda da çalışan bileşen. <code>"use client"</code>            |
| paketleyici      | <i>bundler</i> — dosyaları çevirip birleştiren program              |
| daraltma         | <i>narrowing</i> — TypeScript'in akışa bakarak tipi kesinleştirmesi |
| ortam değişkeni  | Kodun dışında duran ayar. <code>process.env</code>                  |
| sunucusuz        | <i>serverless</i> — istek geldiğinde çalışıp kapanan kod            |

## — On iki tuzak

TUZAK

BELİRTİ · ÇÖZÜM

|                                            |                                                                        |
|--------------------------------------------|------------------------------------------------------------------------|
| Gizli anahtarı commit'lemek                | Geçmişte kalır. <b>Anahtarı iptal et, yenisini al</b>                  |
| <code>node_modules</code> 'ü commit'lemek  | Her klonda indirilir. <code>git rm -r --cached</code>                  |
| Yanlış klasörde <code>git init</code>      | Masaüstü depo olur. <code>.git</code> 'i sil · önlem: <code>pwd</code> |
| GitHub deposunu README ile açmak           | <code>unrelated histories</code> · depoyu boş aç                       |
| <code>git add</code> 'den sonra düzenlemek | Yeni hâli commit'e girmez · tekrar <code>git add</code>                |
| Push'ta şifre yazmak                       | GitHub 2021'de kapattı · <b>token</b> üret                             |
| <code>-m</code> olmadan commit             | vim açılır · <code>:q!</code>                                          |
| <code>"type": "commonjs"</code> bırakmak   | Modül biçimi hatası · <b>satırı sil</b>                                |
| Commit'leyip push etmemek                  | Canlı değişmez · <code>git status</code> ilk satır                     |
| Panele tırnaklı değer yapıştırmak          | <code>authentication failed</code> · tırnaksız + redeploy              |
| JSX içinde <code>//</code>                 | Yorum ekranda görünür · <code>{/* */}</code>                           |
| Sayfanın statik kalması                    | Veri donar · <code>force-dynamic</code>                                |

## — Cevaplar

Cevaba bakmadan önce kendi cümleleri kur. Yarıları doğruysa asıl öğrenilecek yer **yanlış olan yarıları** — kitabın çıktığı projede en çok orada ilerlendi.

### Bölüm 01 — Terminal ve Git

01 · 1

`git add index.html` yazdın, sonra aynı dosyada bir satır daha değiştirdin ve `git commit` attın. Son değişiklik commit'e girdi mi?

Girmedi. `add` bir işaret koymuyor, dosyanın **o andaki içeriğinin kopyasını** alıyor. Sonraki düzenleme yeni bir içerik — tekrar `git add` gerekiyor. `git status` böyle bir durumda aynı dosyayı **iki bölümde birden** gösterir.

## GitHub yarın kapansa projen gider mi?

Gitmez. Git **dağıtık**: `.git` klasöründe bütün geçmişin tam kopyası senin diskinde duruyor. GitHub sadece bu depoları barındıran popüler bir ev sahibi — Git'in parçası değil.

`.env.local` 'i yanlışlıkla commit'ledin, fark edip `.gitignore` 'a ekledin. Şifren güvende mi?

Değil. `.gitignore` sadece **takip edilmeyen** dosyaları gizler ve geçmişi temizlemez; şifre eski commit'in içinde duruyor. Tek doğru hamle: **anahtarı iptal et, yenisini al**.

## Bölüm 02 — Node, npm ve paketler

`node_modules/` klasörünü sildin, proje çalışmaz oldu. Bu klasör depoya girmiyorsa nasıl geri geliyor?

`npm install` ile. Liste `package.json` 'da, hangi sürümün kurulduğu `package-lock.json` 'da yazıyor. İkisi depoda olduğu için klasör her makinede **birebir** yeniden kurulabiliyor.

**typescript** neden `devDependencies` 'te ama `@neondatabase/serverless` `dependencies` 'te?

Ölçüt tek soru: **canlıdaki sunucu buna çalışırken ihtiyaç duyuyor mu**. TypeScript sadece derleme ve kontrol sırasında lazım, derleme bitince yerde kalıyor. Veritabanı sürücüsü ise her istekte çalışıyor.

Tarayıcıda `document.title` yazınca cevap alıyorsun, Node'da yazınca hata. Dil mi farklı?

Dil aynı, **ortam** farklı. `document` ve `window` tarayıcının verdiği araçlar. Node'da tarayıcı yok; onun yerine dosya okuma, ağ dinleme gibi tarayıcının vermediği araçlar var.

## Bölüm 03 — TypeScript

`npm run dev` tip hatasında durmuyor, `npm run build` duruyor. Bu neden kasıtlı?

Yazarken her yarım satırda durmak işi keser — o yüzden geliştirme sunucusu sadece uyarır. Asıl tehlike bozuk tiple **canlıya çıkmak**, o yüzden kapı üretim derlemesinde.

`process.env.DATABASE_URL` 'in tipini TypeScript neden `string | undefined` sayıyor?

Çünkü `.env` dosyasının içeriğini bilmiyor; ortam değişkeni çalışma anında geliyor. `if (!url) throw ...` yazdığında akışı takip edip **altta artık dolu olduğunu** anlıyor — buna *daraltma (narrowing)* deniyor.

## Tip bilgisi canlıdaki koda hiç girmiyorsa TypeScript sana ne kazandırıyor?

Hatayı **çalıştırmadan önce** yakalamayı. `entry.errors / entry.wordCount` gibi bir satır çökmeden, sessizce yanlış sayı üretir; tip kontrolü onu sen yazarken söyler.

## Bölüm 04 — Next.js'in iskeleti

`page.tsx` 'i hiçbir yerden import etmiyorsun, yine de sayfa ekrana geliyor. Onu kim çağırıyor?

Next.js. Klasör ağacını okuyup **çağırın kodu kendisi üretiyor** ve `.next/` klasörüne yazıyor. Klasik React'te o satırı sen yazıyordun; burada senin elinle yazacağın kod artık üretiliyor.

`export default function Sayfa()` — adını `Sayfa` koydun, çalıştı. Peki `export const metadata` yerine `export const sayfaBilgisi` yazsan?

Çalışmaz — hata da vermez, **sessizce görmezden gelir**. Kural: `export default` dosyada tek olduğu için adına bakılmaz; isimli export'ta aranan **adın** kendisidir.

Aynı üst bar üç sayfada duruyor ve sekme değiştirince yanıp sönmüyor. Bunu hangi dosya sağlıyor?

`layout.tsx`. Kabuk yerinde kalıyor, sadece `{children}` 'ın oturduğu yer yeniden çiziliyor. Üç dosyaya kopyalasaydın logo değişince üç yeri düzeltmen gerekirdi.

## Bölüm 05 — Next.js — tam harita

`app/kayıt/[id]/page.tsx` — 142 kayıt için kaç dosya yazacaksın?

Bir tane. Köşeli parantez **o konumdaki her değeri** kabul ediyor; hangi değer geldiği sayfaya parametre olarak veriliyor. `/kayıt/42` ile `/kayıt/117` aynı dosyaya düşüyor.

Bir dosyanın en üstüne `"use client"` yazdın. O dosya `db.ts` 'i import ediyor. Ne oldu?

`db.ts` de tarayıcıya indi — `"use client"` bir düğme değil, **bir sınır**; sınırın altındaki her şey iniyor. Şifren artık herkeste. Doğrusu: sınırı yaprağa yakın tut, sadece düğmenin kendisini işaretle.

Sayfa veritabanından saat gösteriyor ama saat hep aynı kalıyor. Teşhis?

Sayfa **statik** kalmış: sorgu derlemede bir kez çalışmış, sonuç metne gömülmüş. Kullanıcı verisi gösteren sayfa dinamik olmalı — `export const dynamic = "force-dynamic"`.

## Bölüm 06 — Sunucu tarafı ve veritabanı

06 · 1

**Server Component'e koyduğun `console.log` nerede çıkar — ve bu neden bütün mimariyi özetler?**

Terminalde, tarayıcı konsolunda değil. Çünkü fonksiyon **sunucuda** çalıştı; tarayıcıya sadece ürettiği HTML gitti. Kodun kendisi hiç inmedi.

06 · 2

**Klasik React'te sayfanın kaynağında `<div id="root">` boş görünürdü, şimdi içerik dolu. Kullanıcı için pratik farkı ne?**

İlk açılışta boş ekran ya da dönen çember yok — içerik **ilk karede** orada. Arama motorları da metni doğrudan görüyor, JavaScript'in çalışmasını beklemiyor.

06 · 3

**`NEXT_PUBLIC_` ön ekini koymadıkça ortam değişkeni tarayıcıya gitmiyor. Bu ön ek neyi zorunlu kılıyor?**

**Bilinçli onayı.** Sızdırmak için fazladan bir şey yazman gerekiyor; kazara olmuyor. Ön eki gördüğün an "bu değer herkese açık" diye okuyabiliyorsun.

## Bölüm 07 — Yayın

07 · 1

**Commit attın, Vercel hâlâ eski sayfayı gösteriyor. İlk nereye bakarsın?**

`git status` 'ün ilk satırına. `ahead 2` yazıyorsa iki commit yerelde duruyor demektir. Vercel senin diskini değil, **deponu** izliyor: `git commit` ≠ yayın, `git push` = yayın.

07 · 2

**`.env.local` 'de `DATABASE_URL="postgres://..."` yazıyor. Aynısını tırnaklarıyla Vercel paneline yapıştırırsan ne olur?**

Tırnaklar değerın parçası sayılır ve şifre yanlış olur — `password authentication failed`. Dosya biçiminde tırnak gerekir, panelin form kutusunda gerekmez. **Aynı metin, iki farklı kural.**

07 · 3

**`localhost:3000` sende açılıyor, arkadaşında açılmıyor. Neden?**

`localhost` "bu bilgisayar" demek — onun makinesinde senin sunucun yok. Herkesin görmesi için 7/24 açık, adresi ve HTTPS sertifikası olan, Node çalıştırabilen bir makine gerekiyor.